

group-tag-logic

October 21, 2025

```
[ ]: # =====
# TAG GROUPING STARTUP CONTEXT
# =====

# This section describes the startup state of the tag grouping subsystem.

# When FastAPI starts:
#   - Configuration is loaded from config.py:
#       GROUP_TAG_THRESH = 0.85 (semantic similarity threshold)
#   - Core services are initialized:
#       + EmbeddingManager - responsible for encoding tag text into vectors.
#       + TagManager       - assigns tags to semantic groups.
#       + GroupManager     - orchestrates grouping and UI updates.
#       + tagset_store     - JSON-based persistent store for grouped tags.
#
# The grouping subsystem remains idle until a new topic arrives and tag
↳ embeddings are generated.

'''text
System Startup State:
↓
Configuration loaded (GROUP_TAG_THRESH)
↓
TagManager + GroupManager initialized
↓
System idle - waiting for new MQTT message with tags
'''
```

```
[ ]: # =====
# NEW TOPIC ARRIVAL
# =====

# When a new MQTT message arrives:
#   topic = "building1/floor2/temp_sensor_A"
#   payload = { "fields": {"value": 23.4}, "tags": {"floor": "2", "room":
↳ "officeA"} }
#   timestamp = 1732041036
```

```

#
# The TopicHandler extracts:
#   → topic name
#   → tag dictionary
#   → timestamp
#
# The tag dictionary is passed to EmbeddingManager for tag embedding.

# EmbeddingManager:
#   - Normalizes each tag value (lowercase, replace "_" with space).
#   - Converts all tag values into semantic embeddings using BAAI/
  ↳ bge-small-en-v1.5
#   - Once embeddings are ready:
#       → Calls groups_manager.update_for_topic(topic, tags, tag_embeddings)
'''text
Topic Arrival Flow:
↓
MQTT message received → extract topic, tags, and timestamp
↓
Tag values sent to EmbeddingManager for encoding
↓
EmbeddingManager normalizes tags and generates vectors
↓
Embeddings passed to GroupManager for grouping process
'''

```

```

[ ]: # =====
# GROUP MANAGER - TAG PROCESSING
# =====

# GroupManager.update_for_topic(topic, tags, embeddings):
#   - Iterates through each tag-value pair and corresponding embedding.
#   - For each tag:
#       → Calls tagset_store.find_or_create_set(tag, embedding, self.
  ↳ threshold, topic)
#   - Uses threshold = config.GROUP_TAG_THRESH for all similarity checks.
#
# After all tags processed:
#   - Collects sets that now include two or more topics.
#   - If valid sets exist:
#       → Broadcast updates to frontend through WebSocket.
#
'''text
Flow Summary:
↓
New tag embeddings received
↓

```

```

Each tag → checked against existing tag sets
↓
Merge if similar threshold / create new set otherwise
↓
Broadcast updated tag groups to UI
'''

```

```

[ ]: # =====
# TAG MANAGER - TAG GROUPING DECISION
# =====

# TagManager.process_tag(tag, embedding, topic):
#
# Step 1 - Check if tag already exists in any stored set:
#     if tag in s["tags"]:
#         tagset_store.add_topic_to_set(s["id"], topic)
#         return s["id"]
#
# Step 2 - Compare tag embedding to each existing set centroid:
#     similarity = cosine_similarity(tag_vector, set_centroid)
#     if similarity > threshold (GROUP_TAG_THRESH):
#         add tag to that set
#         update centroid:
#             new_centroid = ((centroid * (n - 1)) + tag_vector) / n
#         link topic to set
#         save tagset_store.json
#         return s["id"]
#
# Step 3 - If no match passes threshold:
#     create new set:
#         id = "set_" + (len(existing_sets) + 1)
#         tags = [tag]
#         centroid = tag_vector
#         topics = [topic]
#     save new set
#     return new_id
#
'''text
TagManager Flow Summary:
↓
Receive tag and embedding
↓
If tag already exists → link topic and return
↓
Else compare with each centroid using GROUP_TAG_THRESH
↓
If similar → merge tag into set and update centroid

```

```

↓
If dissimilar → create new tag set
↓
Save changes and return set ID
'''

```

```

[ ]: # =====
# TAGSET STORE - DATA STRUCTURE
# =====

# tagset_store.json maintains persistent group data.

# Each record structure:
# {
#   "id": "set_1",
#   "tags": ["floor1", "floorA"],
#   "centroid": [embedding vector],
#   "topics": ["temp/floor1", "humidity/floorA"]
# }

# Stored sets are dynamically updated when new tags merge or new sets are
↳ created.
# Centroids evolve as more similar tags are added.

# Helper functions:
#   tagset_store.add(new_set)           → add new tag group
#   tagset_store.add_topic_to_set(id, t) → link topic to set
#   tagset_store.save()                 → persist to JSON file

'''text
Tagset Store Overview:
↓
Each tag set record stores ID, tags, centroid, and topics
↓
When new tags merge, centroid is recalculated
↓
All updates persisted in tagset_store.json
↓
Provides persistent semantic tag grouping history
'''

```

```

[ ]: # =====
# WEBSOCKET BROADCAST TO FRONTEND
# =====

# Once valid tag sets (sets with 2 topics) are updated:
#   GroupManager broadcasts grouped tag info to all WebSocket clients:

```

```
#
# ws_manager.broadcast({
#     "event_type": "group",
#     "data": { "sets": valid_sets }
# })
#
# Example message:
# {
#     "event_type": "group",
#     "data": {
#         "sets": [
#             {"id": "set_1", "tags": ["floor1", "floorA"]},
#             {"id": "set_2", "tags": ["room201", "office201"]}
#         ]
#     }
# }
#
'''text
Broadcast Flow:
↓
GroupManager → WebSocketManager.broadcast()
↓
Frontend "Tag Grouping Panel" updates instantly
↓
Users see live semantic tag clusters
'''
```

```
[ ]: # =====
# CONTINUOUS GROUP EVOLUTION
# =====

# As more topics arrive:
# - Each topic's tags are embedded and processed.
# - TagManager compares them to existing centroids.
# - If similar, merges into an existing set and updates centroid.
# - If new, creates a new tag set.
# - Valid sets (with 2 topics) trigger broadcast updates.
#
'''text
Continuous Grouping Process:
↓
New topics keep arriving via MQTT
↓
Each new tag embedding checked against centroids
↓
Similar tags merged into existing groups
↓
```

```
New sets created for unique tags  
↓  
UI updates in real time via WebSocket  
, , ,
```